MENOUFIA NATIONAL UNIVERSITY

Faculty of Computers & Artificial Intelligence

Smart Voice-Controlled Car System

PROJECT DOCUMENTATION

Course: IoT Protocols

Instructor: Dr. Ahmed Badawey Rasas

Academic Year: 2026 - 2027

PROJECT TEAM

1. Abdallah Waleed Kamal	20230192
2. Saif Sameh Fathy Elsawy	20230142
3. Abdelrahman Ali Ghonemi Ayad	20230174
4. Abdelrahman Fathy Awad Elbahrawey	20230176
5. Mohamed Garib Mohamed Abas	20230301
6. Mohamed Atef Mohamed Ali Shaheen	20230401

1. Introduction

This project presents a Smart Voice-Controlled Car System that allows a robotic car to be controlled using Arabic and English voice commands. The system integrates Artificial Intelligence (Natural Language Processing) with IoT and Embedded Systems to convert human speech into real motor movements.

> ✅ Flutter mobile application is excluded from this documentation as requested.

2. Project Objective

The main goal of this project is to achieve seamless voice interaction with hardware. Specifically, the objectives are:

- Recognize Arabic and English voice commands with high precision.
- Classify the command intent utilizing a dedicated NLP model.
- Convert the detected intent into direct motor control signals.
- Control a physical robotic car remotely over a secure cloud MQTT connection.

3. System Architecture Overview

The system follows an End-to-End flow where data originates from the user and passes through an AI inference engine before reaching the hardware via IoT protocols:

1. **Voice / Text Input** (Captured via User Interface)
2. **NLP Intent Classification Model** (AI Inference)
3. **Flask Backend API** (HTTP Gateway)
4. **Intent-to-Command Mapping** (Logic Layer)
5. **MQTT (HiveMQ Cloud)** (Message Broker)
6. **ESP8266** (Wi-Fi + MQTT Subscriber)
7. **Arduino UNO** (Motor Controller)
8. **DC Motors** (Physical Actuation)

4. Technologies Used

Software Technologies

- Python 3 & Flask (REST API)
- scikit-learn & joblib
- Paho-MQTT
- SpeechRecognition

Cloud & Hardware

- HiveMQ Cloud (Secure MQTT Broker)
- ESP8266 (NodeMCU) & Arduino UNO
- L298N Motor Driver & DC Motors

5. NLP Intent Classification Model

5.1 Purpose of the Model

The NLP model acts as the brain of the system. It converts unstructured spoken or typed commands into predefined intents that explicitly represent car movements, effectively bridging the gap between natural language and machine commands.

EXAMPLE COMMAND (ARABIC)	EXTRACTED INTENT
امشي قدام	`forward`
امشي ورا	`backward`
امشي يمين	`right`
امشي شمال	`left`
اقف	`stop`

5.2 Dataset Structure

The dataset is stored in a structured CSV file ensuring a robust training process:

COLUMN	DESCRIPTION
text	Raw command text collected from various users to capture different dialects.
intent	Target label for classification, mapped to the core movement states.

5.3 Text Preprocessing Pipeline

To maximize accuracy and resilience to varied input formatting, rigorous preprocessing is applied:

- **Lowercasing:** Standardizes English characters.
- **Whitespace Normalization:** Removes trailing and extra inner spaces.

- **Arabic Character Normalization:** Maps variations of Alif (أ, إ, آ) to a standard Alif (ا), Taa Marbuta (ة) to Haa (ه), and Alif Maqsura (ى) to Yaa (ي).
- **Synonym Mapping:** Converts contextual variations (وقف, اوقف, توقف, ستوب) to a uniform command root (اقف).

5.4 NLP Model Code

train_nlp_model.py

```python
# Intent Classification Model for Smart Car Control
import os, sys, re, joblib
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

# Load dataset and clean text
df = pd.read_csv('dataset.csv')
def clean_text(text):
    text = str(text).lower().strip()
    text = re.sub(r"[أإآ]", "ا", text)
    text = re.sub(r"ه" ,"ة", text)
    text = re.sub(r"ي" ,"ى", text)
    text = re.sub(r"(وقف|اوقف|توقف|ستوب)", "اقف", text)
    text = re.sub(r"قدام" ,"قدامي", text)
    text = re.sub(r"ورا" ,"ورى", text)
    return text

df['text_clean'] = df['text'].apply(clean_text)

X_train, X_test, y_train, y_test = train_test_split(
    df['text_clean'], df['intent'], test_size=0.15, random_state=42, stratify=df['intent']
)

# Training pipeline
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(ngram_range=(1,3))),
    ('clf', LogisticRegression(max_iter=2000, C=10, solver='lbfgs'))
])

pipeline.fit(X_train, y_train)

# Save model
os.makedirs('models', exist_ok=True)
joblib.dump(pipeline, 'models/nlp_intent_model.joblib')
```

5.5 Model Performance

The final tuned model exhibits high reliability in intent mapping:

- **Accuracy achieved:** ~93% on the validation set.
- **Robustness:** Highly resilient for both formal Arabic and informal Egyptian commands.
- **Flexibility:** Handles common spelling variations organically via n-gram TF-IDF vectorization.

6. Flask Backend API

6.1 Backend Responsibilities

The Flask server acts as the central intelligence hub bridging the user interface and the hardware endpoint:

- Loads the pre-trained NLP model once into memory at startup for zero-latency inference.
- Receives command text payloads via a robust HTTP POST endpoint.
- Predicts the user's intent using the loaded model, then maps it to a single-character motor command.
- Publishes the resulting command to the HiveMQ MQTT broker.

6.2 Intent to Command Mapping

CLASSIFIED INTENT	TRANSMITTED MOTOR COMMAND (PAYLOAD)
`forward`	**F** (Drive Forward)
`backward`	**B** (Reverse)
`left`	**L** (Turn Left)
`right`	**R** (Turn Right)
`stop`	**S** (Halt Motors)

6.3 Flask API Integration

app.py

```python
# Flask API for Smart Car Control
from flask import Flask, request, jsonify
import joblib, re, ssl
import paho.mqtt.client as mqtt

app = Flask(__name__)

INTENT_TO_COMMAND = { "forward": "F", "backward": "B", "left": "L", "right": "R", "stop": "S" }

# Secure MQTT Setup
client = mqtt.Client()
client.username_pw_set("<USERNAME>", "<PASSWORD>")
client.tls_set(cert_reqs=ssl.CERT_REQUIRED)
client.connect("<HiveMQ_URL>", 8883)

model = joblib.load('models/nlp_intent_model.joblib')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    text = data.get('text', '')

    clean_cmd = clean_text(text) # Custom cleaning func
    intent = model.predict([clean_cmd])[0]
    command = INTENT_TO_COMMAND.get(intent, 'S')

    client.publish("car/control", command) # Dispatch to Broker

    return jsonify({ 'input': text, 'intent': intent, 'command': command })
```

7. MQTT Communication (HiveMQ Cloud)

The IoT transport layer utilizes the MQTT protocol, known for lightweight and fast message delivery designed for constrained devices.

- **Security:** All communication is encrypted via TLS over port 8883.
- **Routing & Efficiency:** The primary channel used is the `car/control` topic. The payload is optimized to a single ASCII character, minimizing bandwidth.

8. Hardware: ESP8266 & Arduino Architecture

8.1 ESP8266 & Arduino Controllers

The ESP8266 connects to the local Wi-Fi, authenticates with HiveMQ, subscribes to `car/control`, and forwards received characters via Hardware Serial to the Arduino UNO, which then drives the L298N Motor Driver.

ESP8266 Snippet

```
void callback(char* topic, byte* payload,
unsigned int length) {
    // Push raw command byte to Arduino
    Serial.write((char)payload[0]);
}

void loop() {
    if (!client.connected()) {
        client.connect("ESP8266", "user",
"pass");
        client.subscribe("car/control");
    }
    client.loop();
}
```

Arduino Snippet

```
void loop() {
    if (Serial.available()) {
        char cmd = Serial.read();
        if (cmd == 'F') forward();
        else if (cmd == 'B') backward();
        else if (cmd == 'L') left();
        else if (cmd == 'R') right();
        else stopMotors();
    }
}
```

9. System Deployment & Usage

9.1 Hardware Assembly Sequence

1. Upload Arduino motor control code to the board.
2. Connect DC motors to the L298N motor driver module.
3. Connect ESP8266 TX/RX pins to Arduino RX/TX via level shifters/resistors.
4. Power motors using a dedicated external power supply to prevent board resets.
5. Verify MQTT connectivity via the serial monitor.

> **Critical Deployment Note:** The default backend server URL utilizes Ngrok (`https://ungraduating-kaylee-nakedly.ngrok-free.dev`). This is an ephemeral link and must be updated in the client application whenever the Flask server is restarted.

10. Mobile Client Context (Flutter)

While the Flutter application source code is excluded from this documentation, its role in the ecosystem is critical. The app provides the physical interface for the user, capturing voice streams via `ar_SA` configured speech-to-text engines.

Supported Voice Commands Catalog

Movement Commands

- "تقدم" / "امشي"
- "تراجع" / "ارجع للخلف"
- "يمين" / "اتجه يمين"

Future Expansion (Speed)

- "أسرع" / "زود السرعة"
- "أبطأ" / "قلل السرعة"

- "يسار" / "اتجه يسار"
- "قف" / "توقف"

11. Conclusion & Results

The project successfully demonstrates a highly functional, real-world application of Artificial Intelligence intersecting with IoT embedded systems. Testing indicates:

- The robotic vehicle responds instantly and accurately to varied Arabic and English voice commands.
- The NLP classification engine achieves a steady ~93% accuracy rate on out-of-sample data.
- The integration over the HiveMQ cloud ensures scalable, real-time wireless operation over the internet, allowing command transmission from anywhere in the world.

12. Future Improvements

- **Manual Override Mode:** Adding an on-screen joystick to the mobile client for traditional control.
- **Granular Speed Control:** Utilizing PWM pins to allow voice commands like "drive slowly".
- **Obstacle Avoidance:** Integrating Ultrasonic sensors to prevent collisions regardless of user intent.